# Fundit V2: crowdfunding escrow on Arc

Version 2.0 draft, October 2026. Prepared for the Circle Developer Grants program.

## Abstract

Fundit is a crowdfunding app where each fundraiser is its own escrow contract on Arc, Circle's stablecoin-native blockchain. Donations arrive in USDC or EURC. The creator withdraws to Arc, or receives USDC on another chain through CCTP V2. The platform operator cannot move or freeze any creator's balance. People sign in with a Fundit account only: a passkey smart account from Circle Modular Wallets. This sign-in ships in the next release. Fundit runs as a static web app with no backend, no database and no server that holds spending keys. The contracts are deployed on Arc mainnet and Arc testnet. This paper explains the design, the trust model and the plan for launch.

## 1. Problem

Online fundraising platforms hold donor money in their own accounts before they pay it out. That model has three costs for the people who use it:

- **Custody risk.** The platform can freeze, delay or lose funds. Donors cannot check where their money is.
- **Slow and local payouts.** Creators wait days for a bank transfer, and cross-border payouts add fees and delays.
- **Opaque books.** Donors see a total on a web page. They cannot audit what came in and what went out.

Earlier on-chain fundraising tools fixed some of this but added new problems. Many pooled funds in an operator vault, relied on bridges with their own trust assumptions, or needed a backend that held private keys. The first version of Fundit had these flaws. It sent donations to an owner vault, kept deposit-wallet keys in a database and paid creators off chain. Fundit V2 is a full rebuild that removes all of it.

## 2. Design goals

1. **Zero custody.** Funds sit in a per-campaign contract. No operator key can move a creator's balance.
2. **Stablecoins only.** USDC and EURC, so a donation keeps its value between giving and payout.
3. **One home chain.** Every campaign lives on Arc. Donors and creators on other chains connect through Circle's cross-chain products, never through a third-party bridge.
4. **No servers with keys, no paid infrastructure.** The app is static. All reads come from public RPC endpoints and on-chain events, plus Circle's public Iris API.
5. **No wallet app needed.** A person signs in with a passkey (a fingerprint, face or device PIN).
6. **Honest claims.** Every test that moves money runs on Arc testnet first.

## 3. Why Arc and Circle

Arc is a Layer 1 blockchain from Circle where USDC is the gas token. Donors and creators hold one asset and never need a separate token for fees. Arc mainnet (chain ID `5042`) is a CCTP V2 domain (`26`), and EURC is live on it. Arc testnet (chain ID `5042002`) has the same CCTP domain and works the same way. Only the token value differs.

Fundit uses these Circle and Arc products:

| Product | Role in Fundit |
|---|---|
| Arc | Home chain for every campaign escrow. |
| USDC and EURC | The two campaign tokens, used through the ERC-20 interface with 6 decimals. |
| Modular Wallets | The Fundit account: a passkey smart account. It is the only way to sign in. |
| CCTP V2 and the Forwarding Service | USDC payouts and account sends to other chains, in one transaction. |
| Bridge Kit | **Add money** from another chain, behind the `VITE_CROSSCHAIN` flag. |

## 4. System overview

Fundit has two parts:

- **Contracts.** `CampaignFactory` and `Campaign`, written in Solidity and tested with Foundry.
- **App.** A React and TypeScript single-page app, live at `fundit.fun` and served as static files from Cloudflare Pages.

```
Connected wallet on Arc, or USDC on another chain (Add money)
                                |
                                v
        Donor's Fundit account (passkey smart account on Arc)
                                |
             approve + donate(), one user operation
                                v
    Campaign escrow on Arc (one EIP-1167 clone per fundraiser)
                                |
    withdraw(): Arc transfer, or CCTP V2 burn with forwarding
                                v
                            Creator
```

### 4.1 Factory

`CampaignFactory` deploys each campaign as an EIP-1167 minimal clone with CREATE2. The salt is the creator's address and a per-creator nonce, so the app can predict a campaign address before it exists. The factory fixes the USDC, EURC and `TokenMessengerV2` addresses in its constructor. The owner cannot add tokens, which keeps fee-on-transfer and rebasing tokens out of the accounting.

At creation the factory copies the current donation fee and withdrawal fee into the new campaign. Later fee changes apply only to campaigns created after the change.

### 4.2 Campaign escrow

Each `Campaign` holds one token and tracks four counters:

- `grossDonated`: donation amounts before fees, tips excluded.
- `creatorBalance`: the net amount the creator can withdraw.
- `feesAccrued`: donation fees, withdrawal fees and tips not yet claimed.
- `withdrawn`: the total the creator has taken out.

The contract keeps this invariant at all times:

```
token.balanceOf(campaign) == creatorBalance + feesAccrued + unaccounted
```

`unaccounted` is any campaign token that arrived without a `donate` call: a CCTP mint or a plain transfer. `settle()` books it as a gross donation and charges the donation fee. `donate`, `withdraw` and `claimFees` settle first, so every inflow pays the same fee no matter how it arrived. The invariant is checked by Foundry invariant tests. Because `unaccounted()` saturates at zero, this is a solvency check: the balance is at least `creatorBalance + feesAccrued`.

`raised()` returns `grossDonated` plus any unaccounted amount. Withdrawals never reduce it, so the progress bar always shows the total raised.

### 4.3 Fundit accounts

A Fundit account is a Circle Modular Wallets smart account on Arc. The owner signs with a passkey bound to `fundit.fun`. Each action is a user operation sent to Circle's bundler. There is no paymaster, so the account pays its own gas in USDC, and every action that spends USDC leaves 0.05 USDC in the account for the network fee.

The account offers these flows:

- **Recovery key.** After sign-up, the person saves 12 words. The account registers the key's address as a recovery address. The words never leave the browser.
- **Recover.** On `/recover`, the 12 words add a new passkey on the current device as an owner.
- **Add money.** The person sends USDC or EURC on Arc to the account address, or sends from a connected wallet. The app switches that wallet to Arc before it sends. With `VITE_CROSSCHAIN` on, Bridge Kit also burns USDC on another CCTP chain with the account as the recipient, and the Forwarding Service mints it on Arc.
- **Send money out.** The account sends USDC or EURC to any address on Arc, or USDC to another CCTP chain with the Forwarding Service.

Today, connecting a wallet in **Add money** is the only use of Reown (WalletConnect). Nobody signs in with a connected wallet.

### 4.4 Donations

The donor's Fundit account approves the campaign for `amount + tip` and calls `donate(amount, tip, message)` in one user operation. The campaign records the account as `msg.sender`.

**Give anonymously** puts the ASCII marker `[anon]` at the start of the message. Fundit pages then show "Anonymous" in place of the donor's name. This hides the name on Fundit only. The gift is not private: the `Donated` event still records the donor's account address on chain.

### 4.5 Withdrawals and payouts

Only the creator can call `withdraw(amount, destDomain, recipient, maxFee, minFinalityThreshold, hookData)`. The organiser can withdraw to Arc or to any chain in the app's CCTP list. The contract settles, takes the withdrawal fee, then:

- On Arc (domain `26`), transfers the net amount to the recipient.
- On another CCTP domain, approves exactly the net amount to `TokenMessengerV2` and calls `depositForBurnWithHook` with the Forwarding Service hook data. Circle mints on the destination chain, and the creator pays no gas there.

The app quotes the Fast and Standard fees from Circle's CCTP fees API before it builds the call. A fork test confirmed a USDC withdrawal to Base (domain `6`) through the real `TokenMessengerV2` on Arc mainnet with plain `depositForBurn`. The hook path is covered by unit tests against a mock messenger.

### 4.6 EURC campaigns

EURC campaigns take donations and pay out on Arc only. On Arc mainnet, CCTP cannot burn EURC today. On a fork, `depositForBurn(EURC)` to domains 0, 1, 6 and 14 reverts with "Burn token not supported", and `TokenMinterV2.burnLimitsPerMessage(EURC)` is 0. Press reports said EURC CCTP was live on Arc. Fundit follows on-chain state instead.

The factory takes a list of EURC payout domains in its constructor, with no setter. The deployed list is empty. A creator can still withdraw EURC on Arc and move it themselves. Adding an EURC route later needs a new factory.

### 4.7 The app

The app reads everything from chain. The home page and explore page read the factory in pages and multicall each campaign's `summary()`. Donation history comes from `eth_getLogs` on the campaign address. Every campaign event ends with `prevActivityBlock`, a pointer to the previous block with campaign events. The app walks backwards in 10,000-block windows, the limit of the public Arc RPC, and skips quiet blocks.

Each transaction waits for its receipt before the app shows success. The app estimates gas on Arc and adds a 25% margin. Names come from ENS and Basenames. Campaign images are links, with no uploads.

Hosting is Cloudflare Pages. A content security policy limits scripts to the site itself and network calls to the Arc RPC, the CCTP chain RPCs, Circle's APIs and the WalletConnect hosts that **Add money** needs.

## 5. Trust model

### 5.1 What the owner can do

The factory has one owner, set at deploy, with `Ownable2Step` transfers. The owner can:

- Set the donation fee and withdrawal fee for new campaigns, each at most 5% (`MAX_FEE_BPS = 500`).
- Set the treasury to any non-zero address. `claimFees` pays the treasury named at claim time, so a new treasury also receives unclaimed fees and tips from existing campaigns.
- Set a featured campaign.

- Pause and unpause the creation of new campaigns.

`renounceOwnership` always reverts, so the treasury can always be rotated.

### 5.2 What the owner can never do

- Move, freeze or redirect any creator balance.
- Pause donations, withdrawals, settlement or fee claims on an existing campaign.
- Change an existing campaign's fees, token, creator or goal.
- Add a token, change the messenger or upgrade any contract.
- Take part in creator recovery.

### 5.3 Fees

The deployed configuration is a 1% donation fee and a 1% withdrawal fee. Each fee rounds down in the creator's favor. Tips are optional and chosen by the donor. Fees stay in the campaign until anyone calls `claimFees()`. A treasury that reverts can block only fee claims, never donations or withdrawals.

### 5.4 Creator recovery

A creator who loses their key would lock the campaign balance forever. To prevent that, a creator can name an optional backup address. The backup calls `initiateRecovery`, waits 7 days (`RECOVERY_DELAY`) and then calls `completeRecovery` to become the creator. During the wait the creator keeps every power and can cancel with `cancelRecovery`. Changing the backup also cancels a pending recovery. No recovery function moves tokens.

### 5.5 Wrong-token rescue

If someone sends a token other than the campaign token, the creator can call `sweep(token, to)` to recover it with no fee. The call reverts if the campaign-token balance drops, which guards against a second address that controls the same asset.

## 6. Security process

- **Tests.** Foundry unit, fuzz and invariant tests. The invariants cover the balance equation, token conservation, raised never decreasing, fees within the creation snapshot, stray tokens and a blocked treasury. Unit tests cover owner powers.
- **Fork tests.** An Arc mainnet fork suite runs under Arc Foundry `v0.8.0-2`. It only reads chain state. It covers the CCTP payout path with plain `depositForBurn`, EIP-3009 support and EURC CCTP support.
- **Static analysis.** Slither, with every accepted finding listed in `contracts/SECURITY.md`.
- **Arc testnet.** The app has an Arc testnet build setting. A build with `VITE_ARC_NETWORK=testnet` reads every address and URL for testnet, and refuses to run on `fundit.fun`. The end-to-end suite runs on Arc testnet against the testnet factory. It signs up with a virtual passkey through Circle's real flow and runs each money flow: add money, create, give anonymously, withdraw and send money out. It checks every result on chain.

The contracts have not had a third-party audit. A grant would fund one (section 10).

## 7. Business model

Fundit earns the donation fee, the withdrawal fee and optional tips. All three are visible on chain and in the donate form before the donor signs. The fees for a campaign never change after it is created.

Fundit has no infrastructure bill. The app is static, hosting uses a free tier, and all reads use public RPC endpoints.

## 8. Status

As of October 2026:

- Contracts: deployed on Arc mainnet and verified on Sourcify (exact match). They are not verified on the Arc explorer, which blocks automated verification. The addresses and deploy block are in the table below.
- Contracts on Arc testnet: deployed at block `67870064`. The addresses are in the table below.
- App: `fundit.fun` runs on Arc mainnet. Passkey sign-in ships with the next release, once the `fundit.fun` Circle client key is set. In that release, sign-in is passkey only. Add money from another chain is behind `VITE_CROSSCHAIN`.
- Traction: none yet. Metrics in this paper are placeholders until launch.

Arc mainnet contracts (deploy block `26452579`):

| Contract | Address |
|---|---|
| `CampaignFactory` | `0xf1b2AEF82c37fC41D254D8e06BD7780ce34617d9` |
| `Campaign` implementation | `0x083CE7a68Ae418f832E12c2aB976615237e03853` |
| Owner and treasury | `0x3552E9c14C7b6050839009A5B160A4446a1e916D` |

Arc testnet contracts (deploy block `67870064`):

| Contract | Address |
|---|---|
| `CampaignFactory` | `0x6890D40a9A472D0d05aDb5938D85B07059ad54ab` |
| `Campaign` implementation | `0xC7109CAAA5375EE740b95a3DeE52c4C92A02Ba8E` |
| Owner and treasury | `0x3552E9c14C7b6050839009A5B160A4446a1e916D` |

## 9. Launch plan

1. Done: deploy the factory to Arc testnet and record its address.
2. Done: run the end-to-end specs on Arc testnet with a virtual passkey: sign up, save a recovery key, add money, create a fundraiser, give, withdraw and send money out.
3. Run a manual passkey check on a testnet build.
4. Set the `fundit.fun` client key and release passkey sign-in on `fundit.fun`.

## 10. Roadmap and grant milestones

The milestones below are proposed for a Circle Developer Grant paid in USDC on delivery. Targets in brackets are placeholders to agree with the program.

| Milestone | Deliverable | Evidence |
|---|---|---|
| M1: Mainnet launch | Factory deployed and app live on `fundit.fun` (both done), passkey sign-in released | Arc explorer links, live transactions |
| M2: Add money from other chains | `VITE_CROSSCHAIN` enabled after testnet checks | Live transactions for each source chain |
| M3: Audit | Third-party review of `Campaign` and `CampaignFactory`, fixes in a new factory if needed | Published report |
| M4: Adoption | [placeholder: number of campaigns] campaigns and [placeholder: USDC volume] raised | On-chain factory data |
| M5: V2 card donations | Onramp Kit card donations into campaigns | Live campaign with a card donation |

Later ideas, not milestones yet:

- **Private giving.** Gifts through a zero-knowledge (ZK) pool, so the donor's address is not linked to the gift on chain.
- **Giving agent.** An agent for giving that connects wallets through Reown. No agent will hold a key that controls money.

## 11. Risks

- **Fixed configuration.** The fees for existing campaigns and the EURC payout list cannot change after the deploy. A new EURC route needs a new factory.
- **Passkey domain.** Passkeys are bound to `fundit.fun`. Moving to another domain would give people new passkeys and new account addresses, so the domain must stay.
- **Lost passkey.** A person who loses every device with their passkey and has no recovery key cannot get the account back. The app asks for a recovery key until one is saved.
- **Smart account payouts.** A Fundit account may not exist on other chains. The app warns before a payout to another chain and asks for an address the person controls there.
- **No audit yet.** Mitigated by tests, fork tests, Slither and testnet runs. An audit is milestone M3.

## References

- Contracts: `contracts/src/Campaign.sol`, `contracts/src/CampaignFactory.sol`
- Trust model: `contracts/SECURITY.md`
- Deployment record: `contracts/deployments/arc-mainnet.json`
- Network values: `app/src/config.ts`
- Circle developer docs: https://developers.circle.com